What the Agent Sees Matters More Than Search: Novelty and Objective Search for Neuroevolved Block Builders

Lars Ponikvar
New York, NY
lars.ponikvar@gmail.com

***Abstract*—Novelty search is usually recommended for deceptive problems, where following the objective leads away from the best solution. I tested this on a block building task. Small neural networks place up to 40 blocks in a voxel grid, one at a time, and are evolved without backprop. A structure collapses if its center of mass leaves the base. Building a long bridge is deceptive, since it needs counterweights behind the base that add nothing to the score, while building a tower is not. Random, objective, novelty and a 50/50 blend of objective and novelty selection were compared. With a multilayer perceptron (MLP) that reads the whole grid, objective search got a median bridge of 5.5 out of 11 after 200 generations. Novelty search, with the settings used in this experiment, did not beat objective on any task. Novelty's population mean was about half of objective's on the bridge and two thirds on the tower. The blend had the same population mean as objective, and its lead on best bridge length did not hold up after correcting for multiple comparisons. A second agent with 225 weights, which scores each cell from local inputs including how far the structure leans, reached a median best of 11, and there objective search had the best mean. Without the two lean inputs it did about as well as the MLP. Changing what the agent could see made a much larger difference than changing the search method.**

***Index Terms*—novelty search, neuroevolution, deception, genetic algorithms**

I. Introduction

If the steps toward a goal do not raise the score, a search that only follows the score can get stuck. Goldberg called these problems deceptive [1]. Lehman and Stanley's answer was novelty search, which ignores the objective and rewards behavior that is different from what has been seen before [2], [3]. It beat objective search in their deceptive mazes and biped walking, and the common takeaway since then is that novelty is worth it when a task is deceptive and a waste of time when it is not [4], [5].

This treats deception as something the task has on its own. The same score can be hard for one agent and easy for another though, depending on what the agent can see. To look at this I used a block building task with a balance rule and two different agents. A bridge needs counterweights behind its base before it can reach further, and those counterweights do not raise the score. A tower built in the same grid needs nothing like that and is used as a control.

The hypotheses were: H1, on the bridge objective search stalls well below what is possible. H2, on the bridge novelty search ends up ahead of objective search. H3, on the tower objective search does at least as well as novelty. H4, an agent that cannot see the lean of the structure does much worse on the bridge than one that can.

H1 held at 200 generations, but longer runs showed objective search kept climbing slowly. H2 was not supported. H3 and H4 held. Novelty search had a lower population mean than objective search in every condition, and removing two inputs from the agent changed the outcome more than any change to the selection method. The code and all runs are at https://github.com/laars3/objective-vs-novelty.

II. Related Work

In novelty search an individual is scored by its mean distance to its k nearest neighbors in a behavior space, compared against the current population and an archive of past behaviors [3]. How well it works depends a lot on what that behavior space contains [6]. Cuccu and Gomez found novelty alone fails when the behavior space is too large to cover, and that a linear mix of novelty and fitness does better [7]. Gomes et al. tested many variants and also found mixes of novelty and fitness to be the most reliable [8]. Mouret used novelty as a second objective [9], and Mouret and Doncieux found behavioral diversity helped in several robot tasks [10]. Quality diversity methods go further and keep the best individual in each part of the behavior space [5], [11]–[13].

Such et al. showed that a plain genetic algorithm with truncation selection, Gaussian mutation and elitism can train networks with millions of weights [14], which is the setup used here. Conti et al. added novelty to evolution strategies and the versions that mixed novelty with reward did best on deceptive tasks [15]. Woolley and Stanley found that searching directly for an objective can leave a network with a worse internal representation than open ended search [16].

On building things, Sims evolved bodies and controllers together [17], and Funes and Pollack evolved LEGO structures, including cantilever bridges, using a model of brick statics [18]. EvoCraft moved this into Minecraft, where evolution could grow toward a target shape but could not find a machine that moves [19]. Neural cellular automata grow 2D and 3D structures by running the same small network at every cell on local inputs [20], [21], which is where the idea for the second agent here came from.

Most comparisons of novelty and objective search change the task or the novelty settings and keep the agent the same. Here the task stays the same and the agent changes.

III. Method

A. Environment and Tasks

Each build started from a base block at $(12, 4, 0)$, the center of the floor of a $24 \times 8 \times 12$ voxel grid. Agents were given 40 more blocks to place one at a time, so a structure had at most 41 blocks counting the base. In order for a voxel coordinate to be valid, it had to share a face with an existing block and could not be on the floor. If the center of mass (COM) moved more than half a block from the base in x or y, the block was removed and the build ended. Blocks placed before that point stayed and were scored.

Balance was made a way to fail on purpose, not a rule on which cells are legal. In an earlier version unbalanced cells were masked out, so an agent could never over extend, and objective search reached the maximum. The wrong move has to be possible for a task to be deceptive. Before that, a $12 \times 12 \times 12$ grid was too small. A random network could already reach its max of 5, the final best of every run was the same as the best in generation 0, and a control with mutation turned off behaved the same way, so evolution was only picking the best network from the starting population.

Two tasks used this environment setup. The bridge score was how far the furthest block reached in $+x$ past the base. The tower score was the height of the tallest block. With the base at $x = 12$ and $z = 0$, both had a maximum of 11. Building straight up never moves the COM sideways, so the tower max could be reached with a greedy approach. A bridge (a cantilever in engineering terms) on the other hand needed blocks behind its base in $-x$ to pull the COM back, allowing for blocks further out in $+x$. The counterweights did not increase the score, and extending without them ended the build. The bridge task is therefore deceptive in the sense of Goldberg [1]. A hand built reference bridge reached the max of 11 with 23 blocks counting the base, so the budget alone does not explain a shorter bridge.

B. Agents

At each step an agent scored every legal cell and placed a block in the highest scoring one. A given network therefore always built the same structure. The first agent was an MLP that took the whole grid as an input, one value per cell, and output a score for every cell. It had one hidden layer of 128 ReLU units and approximately 590,000 weights.

The second agent, LocalAgent, scored each valid cell separately, with a single network shared across all cells. Its 12 inputs described that cell: which of its six neighbors were filled, position relative to base, the offset of the current COM from base in x and y (the lean), and the fraction of blocks used. It had one hidden layer of 16 ReLU units and 225 weights. LocalAgentNoCOM was the same network without the two lean inputs, with 193 weights. A random agent that placed each block in any legal cell was used as a baseline.

C. Selection Arms

Every arm used the same genetic algorithm with a population of 50. Each generation every agent built a structure and got a selection score, and the top half survived. The best one was copied over unchanged, and the other 49 slots were filled with copies of random survivors with Gaussian noise added to all their weights. There was no crossover. The noise σ had to be small, 0.002 for the MLP and 0.03 for LocalAgent, which is about 10% of a starting weight. With $\sigma = 0.1$ every child was basically a new random network.

The arms only differed in what they selected on. Objective selected on the task score. Novelty selected on the mean distance to the 15 nearest neighbors in behavior space, out of the current population and the archive [3]. Blend used $0.5\, s/11 + 0.5\, n/n_{\max}$, where s is the score, n the novelty and $n_{\max}$ the highest novelty that generation, so both halves were between 0 and 1. Random used the random agent. Since the kept best agent was the best by the arm's own score, only objective's best score per generation could never drop.

A structure's behavior was summarized with five numbers, all scaled to $[0, 1]$: block count, height, height of the COM, distance of the furthest block from the base, and the average distance of blocks from the base. Each was divided by what is reachable and not by the size of the grid, otherwise a number like block count would sit close to zero for everything. The distances did not have a sign, so reaching behind the base looked the same as reaching in front of it. A second version, used by the noveltysplit and blendsplit arms, split the furthest distance into reach in front and reach behind, so six numbers. A structure went into the archive if its novelty was above a threshold that started at 0.1, went up 5% if more than four were added in a generation, and went down 5% after five generations with none added.

D. Evaluation

For every run the best score and the population mean of each generation were saved. Two numbers were taken from each run, the best score found and the population mean over the last 50 generations (called mean score from here). Arms were compared mainly on the mean score, because the best ever mostly shows how many structures were tried. 200 generations is 10,000 structures, and the random arm got a bridge of 6 just from that.

Runs were 200 generations unless said otherwise. On the bridge, objective, novelty, blend and the two split arms had 30 seeds per agent, random had 10. Tower and LocalAgentNoCOM had 10 seeds per arm. Arms with the same agent started from the same population for the same seed. Medians across seeds are reported. Arms were compared with Cliff's delta δ, which is the chance a run from one arm scores higher than one from the other minus the reverse [22], and a two sided permutation test on δ with 10,000 shuffles. δ is a linear function of the Mann-Whitney U [23], so this is the same test. The 2000 generation runs were small enough for exact permutation tests. All 28 comparisons in this paper were Holm corrected together [24], and every p below is the corrected

TABLE I
MEDIAN ACROSS SEEDS. BEST IS THE BEST SCORE FOUND, MEAN IS THE POPULATION MEAN OVER THE LAST 50 GENERATIONS.

Agent	Arm	Seeds	Best	Mean
Bridge, 200 generations				
MLP	random	10	6	1.02
MLP	objective	30	5.5	3.59
MLP	novelty	30	6	1.97
MLP	blend	30	6	3.60
MLP	noveltysplit	30	6	2.07
MLP	blendsplit	30	7	3.70
LocalAgent	objective	30	11	9.79
LocalAgent	novelty	30	11	5.24
LocalAgent	blend	30	11	9.45
LocalAgentNoCOM	objective	10	6.5	2.92
Bridge, 2000 generations				
MLP	objective	5	8	6.76
MLP	blend	7	9	6.01
Tower, 200 generations				
MLP	random	10	8	2.21
MLP	objective	10	11	8.66
MLP	novelty	10	11	5.47
MLP	blend	10	11	8.93

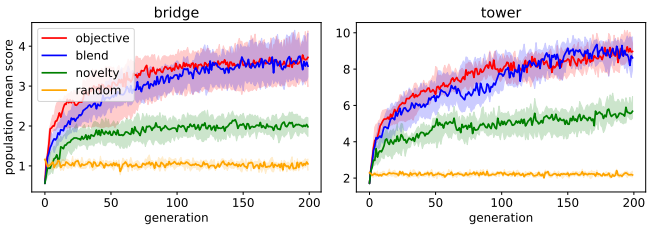


Fig. 1. Population mean score per generation for the MLP, median across seeds with the interquartile range shaded. Bridge has 30 seeds per arm (random 10), tower has 10.

value. With 10,000 shuffles the smallest possible uncorrected p is about 0.0001, so the smallest corrected p is about 0.003.

IV. RESULTS

A. Bridge with the MLP

All results are in Table I. Objective search with the MLP got a median best of 5.5 when 11 is possible, so H1 held at 200 generations. On best score it was no better than random ($\delta = -0.14$, $p = 1.0$). On mean score it was far above it ($\delta = 1.0$, $p = 0.003$), see Fig. 1. On 6 of 30 seeds objective search collapsed and ended at 4 or below. Five of those found their best in the first 10 generations and never improved, and three never beat the best network of generation 0.

H2 was not supported. Novelty and objective were level on best score ($p = 1.0$) and novelty was well behind on mean score, 1.97 against 3.59 ($\delta = -0.89$, $p = 0.003$). Blend had the same mean score as objective ($p = 1.0$). It was ahead of objective on best score ($\delta = 0.38$) and of novelty ($\delta = 0.34$), but neither lead held after correction ($p = 0.12$ and $p = 0.25$). Blend never ended below 5 and got 7 or more on 13 seeds, objective did that on 6.

Fig. 2 shows typical final structures. Objective seed 23 got to 6 by generation 34, with 40 blocks counting the base, and its best structure changed only three times in 200 generations. In novelty seed 22 and blend seed 28 the best structure of the generation changed in 155 and 125 of the 200 generations.

B. Longer Runs

Objective seeds 0 to 4 and blend seeds 0 to 6 were rerun for 2000 generations (Fig. 3), and their first 200 generations matched the short runs exactly. Four of the five objective seeds kept improving after generation 200, the latest at generation 1847, and every gain was one step at a time. Seed 2 stayed at 3 until generation 371 and ended at 7, so the collapses at 200 generations look more like slow starts. Medians ended at 8 for objective and 9 for blend. Blend was a bit ahead on best score ($\delta = 0.34$) and objective on mean score ($\delta = 0.43$), with $p = 1.0$ for both. With differences this size about 25 seeds per arm would be needed, so these runs are only exploratory. H1 only held for a fixed number of generations.

C. What Limits the MLP

In the best MLP structures the blocks behind the base were on average 1.7 to 1.9 blocks behind it, depending on the arm (objective 1.7, novelty 1.75, blend 1.9, averaged over seeds). In the reference they are 5.5 behind. A far counterweight moves the COM more per block, so far counterweights looked like the missing step, and the unsigned behavior summary could not reward them. The split summary was made to test this. The archive got bigger (median 306 entries against 241.5), but noveltysplit was level with novelty and blendsplit with blend ($p = 1.0$ for all three tests). The furthest block behind the base was still about 3.5 behind on average, the same as for blend. Blendsplit was ahead of objective on best score ($\delta = 0.49$, $p = 0.020$), but since it was level with blend this does not show that the split summary helped.

That explanation was wrong. A simple builder that extends the bridge and only adds a counterweight when balance needs one reaches 11 with counterweights at most 3 behind the base in 31 blocks, and at most 2 behind in 38. Close counterweights are enough. The structures point to running out of blocks instead. 13 of 30 objective and 26 of 30 blend best structures used 38 or more of the 41 blocks. Depending on the arm, a median of 6.5 to 11 blocks sat in the same x column as the base, where they add no reach and move the COM less than a block behind the base would. The part in front of the base used 2.2 to 2.6 blocks per step of reach, where a straight arm needs 1. So in 200 generations the limit was how well the MLP used its blocks, and changing the behavior summary did not help with that.

D. Tower

On the tower every arm except random reached a median best of 11. Objective and novelty were level on best height ($p = 1.0$), but on mean score objective was well ahead, 8.66 against 5.47 ($\delta = 0.96$, $p = 0.007$). Blend and objective were level on both ($p = 1.0$). Here objective beat random on best

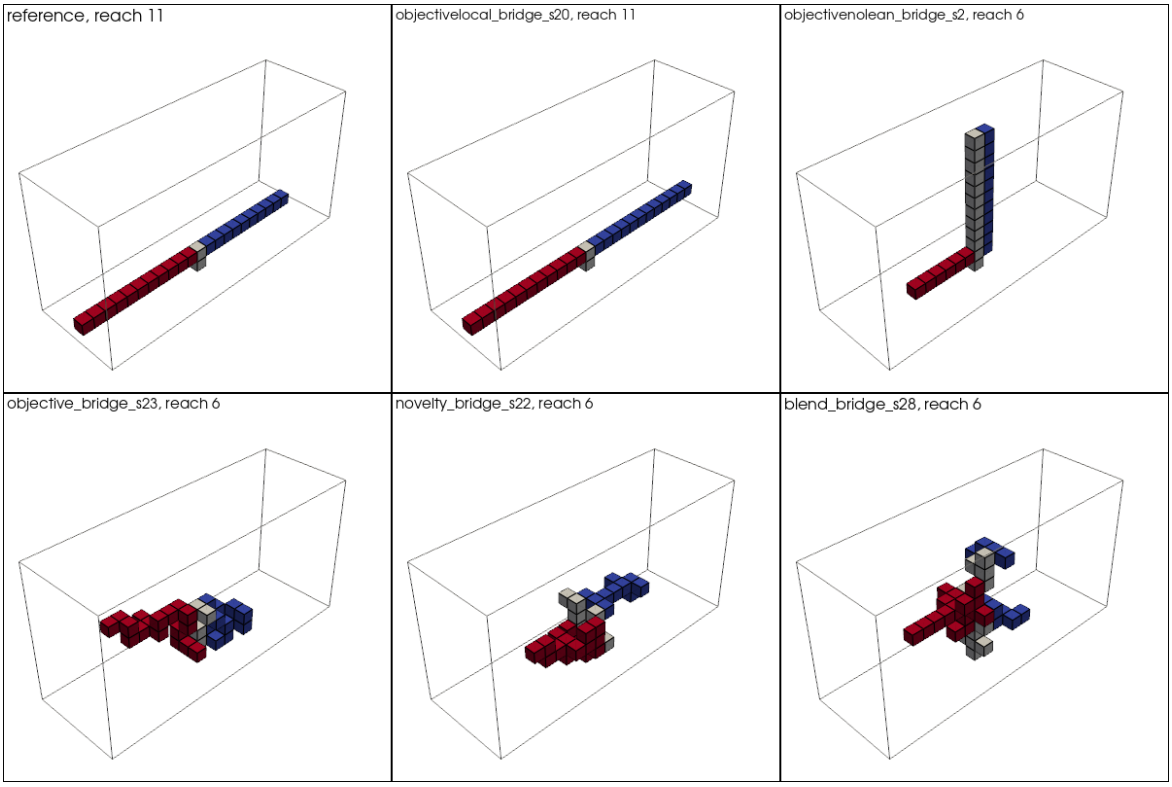


Fig. 2. Best final bridge structures, one seed per condition, picked as the seed closest to the condition's median best and then median mean score. Top: reference (reach 11), LocalAgent objective seed 20 (11), LocalAgentNoCOM objective seed 2 (6). Bottom: MLP objective seed 23, novelty seed 22 and blend seed 28 (all 6). Red blocks are in front of the base, blue behind it, gray above it.

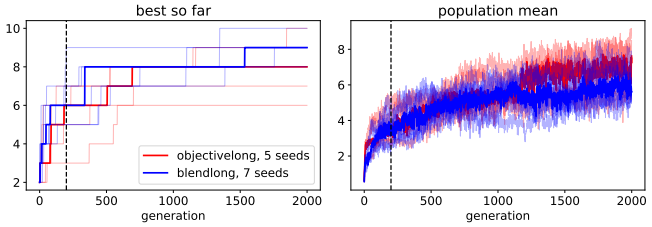


Fig. 3. MLP bridge runs continued to 2000 generations. Thin lines are single seeds, thick lines medians. The dotted line is generation 200.

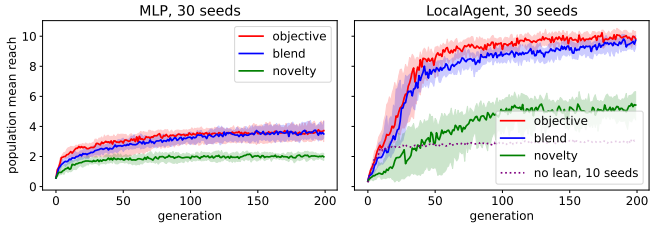


Fig. 4. Population mean bridge reach for the MLP and LocalAgent, 30 seeds per arm, median and interquartile range. The dotted purple line is LocalAgentNoCOM with objective search, 10 seeds, median only.

height ($\delta = 0.98$, $p = 0.006$), which it did not manage on the bridge. H3 held. Novelty's mean was about half of objective's on the bridge and two thirds on the tower, so it was behind on both and making the task deceptive did not change which one won.

E. LocalAgent

With LocalAgent every objective and blend seed reached 11. Novelty reached it on 27 of 30, the other three ended at 7, 4 and 7. The median generation where 11 was first reached was 7.5 for objective, 10.5 for blend and 12 for novelty, and on four seeds (7, 8, 9 and 26) a network in the starting population already built it. Since nearly every seed hit 11 the arms were level on best score, so the mean score is what separated them (Fig. 4). Objective was above novelty on every seed, 9.79 against 5.24 ($\delta = 1.0$, $p = 0.003$), and above blend, 9.79 against 9.45 ($\delta = 0.74$, $p = 0.003$). LocalAgent with objective search also beat the MLP with objective search on both best and mean score ($\delta = 1.0$, $p = 0.003$ for both).

Of the 30 objective best structures, 20 were a flat bridge one or two blocks above the floor, like the reference, using 23 to 41 blocks. The other 10 were a column up to the top of the grid with a beam across the whole grid, which is allowed because balance is only a COM check. With LocalAgent the bridge stopped being deceptive and looked like the tower task. Objective won, and the novelty half of blend only slowed it down.

F. Removing the Lean Inputs

LocalAgentNoCOM with objective search got a median best of 6.5. Only one of the 10 seeds reached 11, at generation 100. Its mean score was 2.92, against 9.91 for LocalAgent on the same 10 seeds (9.79 over all 30). It was behind LocalAgent on best ($\delta = -0.90$) and mean ($\delta = -1.0$), both $p = 0.003$, so H4 held. Against the MLP it was level on mean score ($p = 1.0$) and on best score ($\delta = 0.51$, $p = 0.19$). So most of what LocalAgent gained over the MLP came from seeing the lean, and without it the small shared network did about as well as an MLP with 590,000 weights. Seed 2 is typical: by generation 50 it built a column of about 10 blocks on the base with an arm of 6 and kept that structure to the end.

V. Discussion

Novelty never won, with either agent or on either task. Its best scores were level with objective everywhere, but its population mean was always lower, which makes sense for a method that spreads its population out instead of pushing the score. Time to reach the max does not have that bias, and with LocalAgent novelty was slowest there too, with a median of 12 generations vs 7.5 for objective. No selection change raised the mean score above objective. Blend's lead over objective on best bridge length did not survive the correction, and with LocalAgent the same blend lowered the mean. Earlier work found mixes of novelty and fitness more reliable than novelty alone [7], [8], [15]. With the MLP on the bridge, blend was well ahead of novelty on mean score ($\delta = 0.99$, $p = 0.003$), but only level with objective.

Lean inputs made the biggest difference. One possible reason is that the MLP has to work out from the grid how close a structure is to tipping over. LocalAgent is given the lean as an input, and for it the bridge was about as easy as the tower. Taking those two inputs away dropped it to the MLP's level. So in this setup deception came from the agent and the task together, not only from the score. However, MLP and LocalAgent differ in more than their inputs. The architecture, the number of weights (about 590,000 against 225) and σ all changed at the same time, so the gap between those two agents cannot be put on any one of them. Only lean inputs were tested on their own, through LocalAgentNoCOM. This is close to the point that the behavior summary decides how novelty search does [6] and that how something is searched shapes what it ends up representing [16], except that here it was the agent's inputs that mattered.

The MLP also did not fail the way H1 expected. It was not missing a counterweight that looked useless. It wasted blocks over the base and on thick arms, and over longer runs objective search kept slowly getting better. Changing what novelty rewards cannot fix that, which is why the split summary did nothing. So within the 200 tested generations, the deceptive part of the bridge was not what held the MLP back, which may be why novelty did not help there.

There are limitations. Balance is only a COM check, so a beam on one column counts as balanced, and a collapse keeps the blocks placed before it and those still score. Scores are whole numbers so there are many ties, and selection breaks them by position in the population. Some runs were decided by the starting population: three objective seeds never beat their generation 0 best, and four LocalAgent seeds had an 11 before any evolution. The MLP versus LocalAgent comparison mixes inputs, architecture, number of weights and σ, and only the lean inputs were isolated. 590,000 weights is also a lot to evolve with 50 agents, which may have held the MLP back. The two behavior summaries are close variants of each other, and the novelty settings (k and the archive threshold) were not tuned. Tower and LocalAgentNoCOM had 10 seeds and the long runs 5 and 7. Most runs were 200 generations and the 2000 generation runs were still climbing at the end, so where the arms level off is not known.

Left for future work are a blend weight that changes during a run, other behavior summaries, other tasks, real physics, novelty with an elite kept by reach, MAP-Elites [12], and an MLP given the lean as an extra input, which would separate the inputs from the architecture.

VI. Conclusion

Objective, novelty and blended selection were compared on a deceptive bridge task and a tower task, with two kinds of evolved builder. Novelty search did not win anywhere with the settings used, and the blend did not clearly beat objective search. Giving the local agent two inputs for how far the structure leans let every method reach median best of 11, and taking them out removed most of that. Whether the bridge was deceptive depended on what the agent could see, and for these builders that mattered more than the search method.

References

[1] D. E. Goldberg, "Simple genetic algorithms and the minimal, deceptive problem," in *Genetic Algorithms and Simulated Annealing*, L. Davis, Ed. London: Pitman, 1987, pp. 74–88.

[2] J. Lehman and K. O. Stanley, "Exploiting open-endedness to solve problems through the search for novelty," in *Proc. 11th Int. Conf. Artificial Life (ALIFE XI)*. MIT Press, 2008, pp. 329–336.

[3] ——, "Abandoning objectives: Evolution through the search for novelty alone," *Evolutionary Computation*, vol. 19, no. 2, pp. 189–223, 2011.

[4] K. O. Stanley and J. Lehman, *Why Greatness Cannot Be Planned: The Myth of the Objective*. Springer, 2015.

[5] J. K. Pugh, L. B. Soros, and K. O. Stanley, "Quality diversity: A new frontier for evolutionary computation," *Frontiers in Robotics and AI*, vol. 3, p. 40, 2016.

[6] S. Kistemaker and S. Whiteson, "Critical factors in the performance of novelty search," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO)*, 2011, pp. 965–972.

[7] G. Cuccu and F. Gomez, "When novelty is not enough," in *Applications of Evolutionary Computation (EvoApplications)*, ser. LNCS, vol. 6624. Springer, 2011, pp. 234–243.

[8] J. Gomes, P. Mariano, and A. L. Christensen, "Devising effective novelty search algorithms: A comprehensive empirical study," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO)*, 2015, pp. 943–950.

[9] J.-B. Mouret, "Novelty-based multiobjectivization," in *New Horizons in Evolutionary Robotics*, ser. Studies in Computational Intelligence, vol. 341. Springer, 2011, pp. 139–154.

[10] J.-B. Mouret and S. Doncieux, "Encouraging behavioral diversity in evolutionary robotics: An empirical study," *Evolutionary Computation*, vol. 20, no. 1, pp. 91–133, 2012.

[11] J. Lehman and K. O. Stanley, "Evolving a diversity of virtual creatures through novelty search and local competition," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO)*, 2011, pp. 211–218.

[12] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," arXiv:1504.04909, 2015.
[13] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, "Robots that can adapt like animals," *Nature*, vol. 521, pp. 503–507, 2015.
[14] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," arXiv:1712.06567, 2017.
[15] E. Conti, V. Madhavan, F. P. Such, J. Lehman, K. O. Stanley, and J. Clune, "Improving exploration in evolution strategies for deep reinforcement learning via a population of novelty-seeking agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
[16] B. G. Woolley and K. O. Stanley, "On the deleterious effects of a priori objectives on evolution and representation," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO)*, 2011, pp. 957–964.
[17] K. Sims, "Evolving virtual creatures," in *Proc. 21st Annu. Conf. Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1994, pp. 15–22.
[18] P. Funes and J. Pollack, "Evolutionary body building: Adaptive physical designs for robots," *Artificial Life*, vol. 4, no. 4, pp. 337–357, 1998.
[19] D. Grbic, R. B. Palm, E. Najarro, C. Glanois, and S. Risi, "EvoCraft: A new challenge for open-endedness," in *Applications of Evolutionary Computation (EvoApplications)*, ser. LNCS, vol. 12694. Springer, 2021, pp. 325–340.
[20] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, vol. 5, no. 2, p. e23, 2020.
[21] S. Sudhakaran, D. Grbic, S. Li, A. Katona, E. Najarro, C. Glanois, and S. Risi, "Growing 3D artefacts and functional machines with neural cellular automata," in *Proc. Artificial Life Conf. (ALIFE)*. MIT Press, 2021.
[22] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.
[23] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.
[24] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.